

Introducción a FPGA y Verilog

Curso de Electrónica Digital

May 19, 2026

¿Qué es una FPGA?

FPGA = Field Programmable Gate Array

Es un dispositivo que permite implementar circuitos digitales configurables.

- Implementa hardware real.
- No ejecuta instrucciones como un microcontrolador.
- Permite diseñar desde compuertas hasta procesadores.

¿Cómo se programa una FPGA?

Usando un lenguaje HDL (Hardware Description Language).

Verilog permite describir:

- Conexiones físicas
- Compuertas lógicas
- Sistemas combinacionales
- Sistemas secuenciales

Modelo 1: Descripción con assign

```
module and_gate(  
    input A,  
    input B,  
    output Y  
);
```

```
    assign Y = A & B;
```

```
endmodule
```

Características:

- Modelado conductual simple
- Sintetizable
- Claro y directo

Modelo 2: Compuertas nativas (Primitivas)

Verilog incluye primitivas lógicas:

```
module and_gate(  
    input A,  
    input B,  
    output Y  
);
```

```
and (Y, A, B);
```

```
endmodule
```

Formato general:

```
and (salida, entrada1, entrada2);
```

Otras primitivas disponibles

```
or    (Y, A, B);  
nand  (Y, A, B);  
nor   (Y, A, B);  
xor   (Y, A, B);  
xnor  (Y, A, B);  
not   (Y, A);
```

Estas representan compuertas físicas directas.

Ejemplo estructural con varias compuertas

Función:

$$Y = (A \wedge B) \vee C$$

```
module small_logic(  
    input A,  
    input B,  
    input C,  
    output Y  
);  
wire w1;  
and (w1, A, B);  
or  (Y, w1, C);  
endmodule
```

Aquí usamos:

- wire para conexiones internas
- Descripción estructural

Comparación: assign vs Primitivas

Con assign:

```
assign Y = (A & B) | C;
```

Con primitivas:

```
wire w1;  
and (w1, A, B);  
or  (Y, w1, C);
```

Conclusión:

- assign es más compacto.
- Primitivas muestran estructura física.

Conceptos Clave

- wire conecta señales internas.
- assign describe lógica combinacional.
- Las primitivas representan compuertas físicas.
- Un módulo es un bloque reutilizable.

En esta sesión usaremos:

- Solo compuertas primitivas
- Conexiones con wire
- Instanciación jerárquica

Sistema Ejemplo: Control de Acceso

Abrir puerta si:

$$KEY = 1010 \quad y \quad ENABLE = 1$$

Entradas:

- KEY[3:0]
- ENABLE

Salida:

- OPEN

Módulo 1: Comparador estructural (Parte 1)

Comparar KEY con 1010:

$$MATCH = (K3 = 1)(K2 = 0)(K1 = 1)(K0 = 0)$$

```
module key_compare(  
    input [3:0] KEY,  
    output MATCH  
);  
  
    wire n2, n0;  
  
    not (n2, KEY[2]);  
    not (n0, KEY[0]);  
  
    and (MATCH, KEY[3], n2, KEY[1], n0);  
  
endmodule
```

Análisis del Comparador

Estamos verificando:

- $KEY[3] = 1$
- $KEY[2] = 0$
- $KEY[1] = 1$
- $KEY[0] = 0$

Si todas se cumplen $\rightarrow MATCH = 1$

Comparación estructural real.

Módulo 2: Control de habilitación

```
module enable_control(  
    input MATCH,  
    input ENABLE,  
    output OPEN  
);  
  
and (OPEN, MATCH, ENABLE);  
  
endmodule
```

Solo compuerta AND.

Diseño Jerárquico en Verilog

- Módulos pequeños y reutilizables
- Cada bloque cumple una función específica
- El módulo top integra todo el sistema

key_compare + enable_control $\implies$ Sistema Completo

Módulo Top (Parte 1)

```
module top(  
    input [3:0] KEY,  
    input ENABLE,  
    output OPEN  
);  
  
wire match_signal;  
  
key_compare U1(  
    .KEY(KEY),  
    .MATCH(match_signal)  
);
```

Módulo Top (Parte 2)

```
enable_control U2(  
    .MATCH(match_signal),  
    .ENABLE(ENABLE),  
    .OPEN(OPEN)  
);  
  
endmodule
```

Este módulo es el que se sintetiza en Quartus.

¿Qué es un Testbench?

El testbench:

- Solo para simulación
- No se sintetiza
- Genera estímulos

Testbench (Parte 1)

```
module tb_top;

reg [3:0] KEY;
reg ENABLE;
wire OPEN;

top DUT(KEY, ENABLE, OPEN);

initial begin
    $dumpfile("wave.vcd");
    $dumpvars(0, tb_top);
```

Testbench (Parte 2)

```
KEY = 4'b1010; ENABLE = 1;  
#10;
```

```
KEY = 4'b1001; ENABLE = 1;  
#10;
```

```
KEY = 4'b1010; ENABLE = 0;  
#10;
```

```
$finish;
```

```
end
```

```
endmodule
```

Ahora podemos visualizar la evolución temporal en GTKWave.

Flujo hacia la FPGA

- Diseñar módulos
- Verificar con testbench
- Simular
- Corregir
- Abrir Quartus
- Seleccionar módulo top
- Asignar pines
- Programar FPGA